

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C01: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Geetika T 192511469 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

GEETIKA T

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

1. A robot operates in a grid environment (5×5) . It must move from Start (S) to Goal (G) . Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

1. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2

- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Hospital doctor-to-shift assignment

The hospital has three doctors and three shifts. Each doctor must receive exactly one shift, each shift must be used once, D1 cannot work at night, D3 cannot work in the morning, and D2 must work earlier than D3.

1.1 Formal model

- Variables: D1, D2, D3; each domain is {Morning, Afternoon, Night}.
- All-different: no two doctors may receive the same shift; each doctor receives one shift.
- Unary constraints: $D1 \neq \text{Night}$; $D3 \neq \text{Morning}$.
- Ordering constraint: $\text{rank}(D2) < \text{rank}(D3)$, where Morning = 1, Afternoon = 2, Night = 3.
- Goal test: all three doctors are assigned and every constraint is true.

The all-different condition plus three variables and three shifts means a complete legal assignment is a permutation of M, A, and N. A partial assignment is rejected as soon as it violates a constraint; there is no reason to continue exploring that branch.

1.2 Backtracking search trace

To show the search clearly, I assign doctors in the order $D1 \rightarrow D2 \rightarrow D3$ and try shifts in the order $M \rightarrow A \rightarrow N$. Whenever a choice breaks a rule, I undo it and try the next shift.

Step	Partial assignment / trial	Constraint check	Decision
1	D1 = M	D1 is not assigned N. No duplicate exists.	Keep M.
2	Try D2 = M	M is already used by D1; violates one-doctor-per-shift.	Reject; backtrack D2.
3	Try D2 = A	A is unused. Ordering is not yet fully testable because D3 is unassigned.	Keep A.
4	Try D3 = M	D3 is forbidden from Morning; also M belongs to D1.	Reject; backtrack D3.
5	Try D3 = A	A is already assigned to D2.	Reject; backtrack D3.
6	Try D3 = N	N is unused; D3 is not in Morning; $\text{rank}(D2) = 2 < \text{rank}(D3) = 3$.	Accept complete solution.

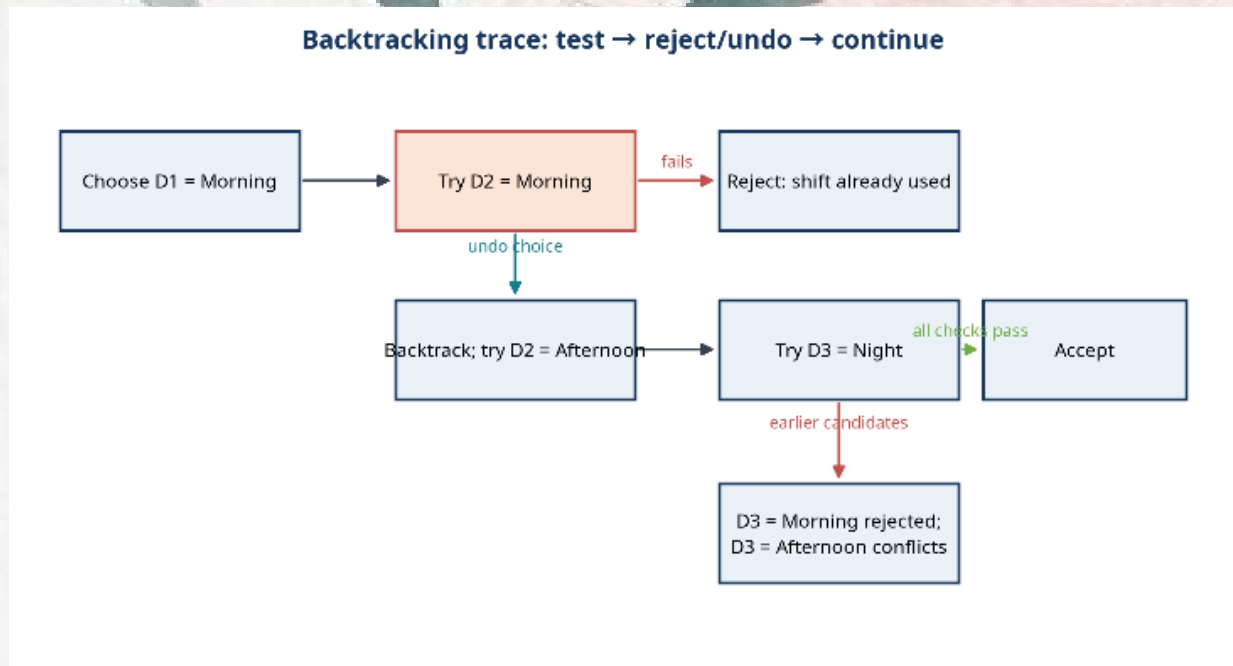


Figure 1. One depth-first branch, with rejected choices visibly undone.

1.3 Final schedule and verification

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

This schedule uses each shift once. D1 is not on Night, D3 is not on Morning, and D2's Afternoon shift comes before D3's Night shift.

Why backtracking fits

This is a small constraint-satisfaction problem: assignments are built incrementally, and illegal partial assignments can be pruned immediately. Backtracking is complete for this finite search space when it explores every remaining value; in the worst case it may examine up to $3! = 6$ complete permutations here, but early constraint checks reduce the actual work.

1.4 Python implementation

```
SHIFTS = ("Morning", "Afternoon", "Night")
```

```
def valid_partial(a):
    # No shift can be used twice.
    if len(set(a.values())) != len(a):
        return False
    if a.get("D1") == "Night":
        return False
    if a.get("D3") == "Morning":
        return False
    # Check the ordering as soon as both doctors are assigned.
    if "D2" in a and "D3" in a:
```

```

    if SHIFTS.index(a["D2"]) >= SHIFTS.index(a["D3"]):
        return False
    return True

def backtrack(order, i=0, assignment=None):
    if assignment is None:
        assignment = {}
    if i == len(order):
        return assignment.copy()
    doctor = order[i]
    for shift in SHIFTS:
        assignment[doctor] = shift
        if valid_partial(assignment):
            answer = backtrack(order, i + 1, assignment)
            if answer is not None:
                return answer
    del assignment[doctor] # undo the trial before trying another value
    return None

answer = backtrack(["D1", "D2", "D3"])
print(answer)
# One valid output: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

```

```

QUESTION 1 - BACKTRACKING
D1 = Morning   ACCEPT: constraints satisfied so far
D2 = Morning   REJECT: shift already assigned
D2 = Afternoon ACCEPT: constraints satisfied so far
D3 = Morning   REJECT: shift already assigned
D3 = Afternoon REJECT: shift already assigned
D3 = Night     ACCEPT: constraints satisfied so far
Final schedule: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}
>>>

```

Figure 2. Direct Python IDLE 3.12.3 Shell capture after running q1_backtracking.py

2. Grid navigation with BFS

Breadth-First Search (BFS) explores a graph one depth level at a time. Because each legal move in Question 2 costs exactly 1, the first shortest route BFS finds is also a minimum-cost route. The prompt does not specify a concrete map, so the following grid is an explicit example rather than information supplied by the question.

Worked map assumptions: a 5×5 grid; $S = (1,1)$; $G = (5,5)$; blocked cells = $(2,2)$, $(2,4)$, $(3,2)$, $(4,4)$; four legal moves (Up, Down, Left, Right), each costing 1. For a reproducible trace, neighbors are considered Right, Down, Left, Up.

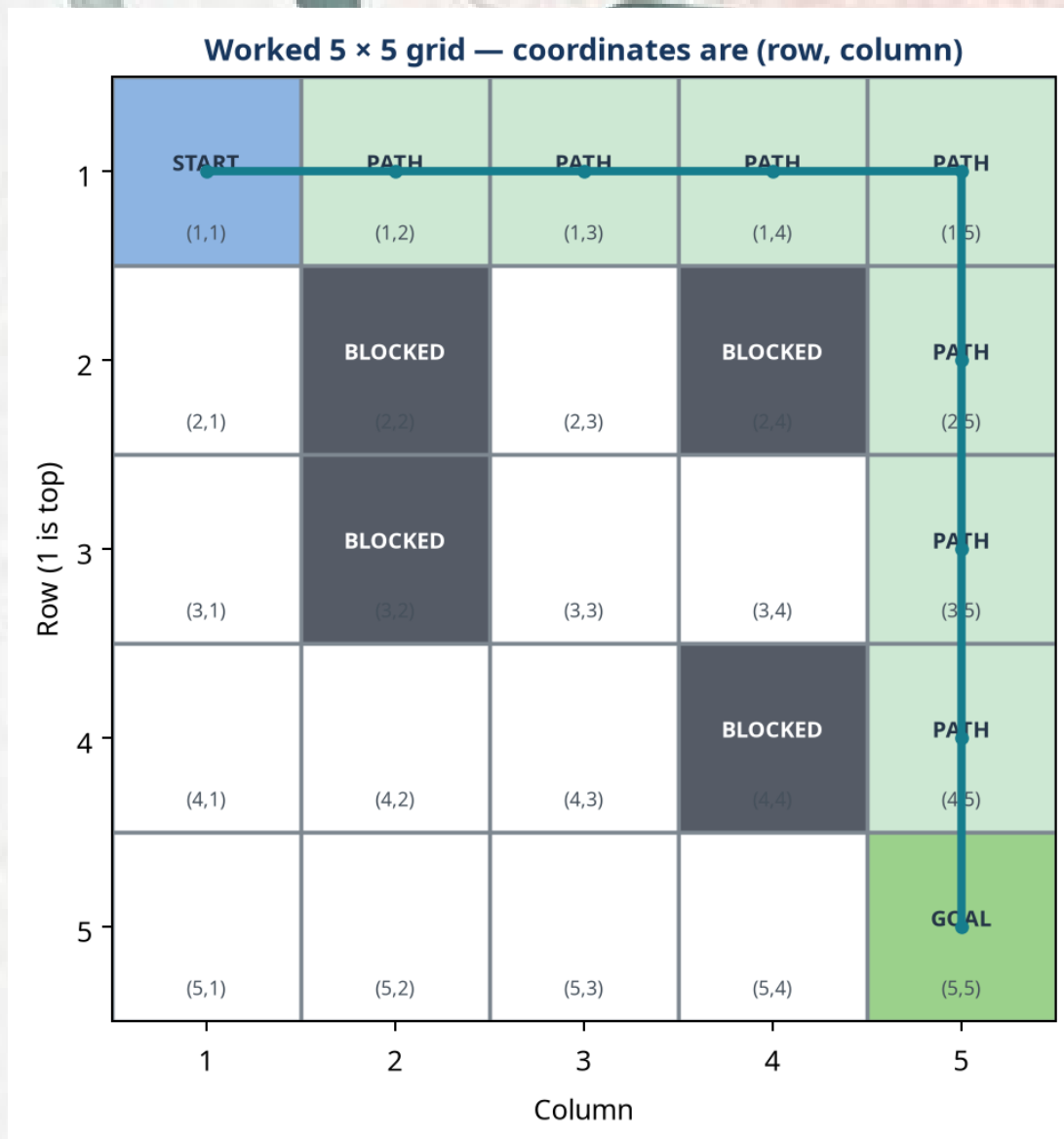


Figure 3. BFS sample map. Green cells mark one shortest route; dark cells are obstacles.

2.1 State representation and Manhattan distance

A state is the robot's current cell (r,c) . A move creates an edge to an adjacent in-bounds cell unless that cell is blocked. The Manhattan heuristic is

$h(r,c) = |r - 5| + |c - 5|$. It ignores obstacles and counts the minimum number of horizontal and vertical steps in an empty grid.

At the start, $h(1,1) = |1-5| + |1-5| = 8$. Since each move costs 1, this is a lower bound on the path cost. The sample map contains a route of exactly 8 moves, so no route can be shorter.

2.2 BFS queue rule

BFS removes the oldest node from a FIFO queue, then appends each newly discovered neighbor to the rear. A visited/parent dictionary prevents repeatedly enqueueing a cell. In this

problem the data structure is a queue—not a priority queue. The h values below are included to honor the heuristic part of the prompt and to show the lower bound at each expanded cell; they do not reorder BFS.

Step	Expanded node (g, h)	New nodes appended at rear	Queue size after update
1	11 (0,8)	12, 21	2
2	12 (1,7)	13	2
3	21 (1,7)	31	2
4	13 (2,6)	14, 23	3
5	31 (2,6)	41	3
6	14 (3,5)	15	3
7	23 (3,5)	33	3
8	41 (3,5)	42, 51	4
9	15 (4,4)	25	4
10	33 (4,4)	34, 43	5
11	42 (4,4)	52	5
12	51 (4,4)	—	4
13	25 (5,3)	35	4
14	34 (5,3)	—	3
15	43 (5,3)	53	3
16	52 (5,3)	—	2
17	35 (6,2)	45	2
18	53 (6,2)	54	2
19	45 (7,1)	55	2
20	54 (7,1)	—	1
21	55 (8,0)	—	0

Notation in the trace: cell 12 means row 1, column 2. “g” is the number of moves from S. The frontier update at each step is: remove the expanded node from the front, then append the listed new nodes in the stated R, D, L, U order. When the goal is removed from the queue at Step 21, BFS stops.

For example, the initial queue is [11]. After expanding 11 it is [12, 21]; after expanding 12 it is [21, 13]. Each expansion removes the front item and appends new neighbors at the rear.

2.3 Reconstructing the path and calculating cost

Each newly discovered node stores its parent. Starting at G, follow parent links backward to S, then reverse the list. For the selected route:

```
(1,1) → (1,2) → (1,3) → (1,4) → (1,5)
      → (2,5) → (3,5) → (4,5) → (5,5)
Move count = 8 edges
Path cost = 8 moves × 1 cost per move = 8
```

Verification: no route cell is blocked; each consecutive pair is one legal orthogonal move; the route ends at (5,5). Since $h(S)=8$ and the found route costs 8, no shorter route can exist.

One shortest route is $S \rightarrow (1,2) \rightarrow (1,3) \rightarrow (1,4) \rightarrow (1,5) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (4,5) \rightarrow G$. It has 8 moves and costs 8.

2.4 Heuristic priority queues: the A* distinction

The wording “BFS” and “priority queue using Manhattan distance” combines BFS with A*. These are not interchangeable. If the instructor specifically expects a priority queue ordered by the heuristic, the technically correct method is A*: each frontier entry is prioritized by $f(n) = g(n) + h(n)$. Because all edge costs are 1 and Manhattan distance is admissible and consistent on a four-directional grid, A* returns an optimal path.

BFS uses a FIFO queue and guarantees the fewest steps when all moves have equal cost; it does not use h to order nodes. A* uses a priority queue ordered by $f = g + h$. On this four-directional unit-cost grid, Manhattan distance is admissible and consistent, so A* also returns an optimal path.

For this particular map, early A* queue updates have $f = 8$ on the optimal corridor. Ties are broken by insertion order (the same R, D, L, U neighbor order), so its early choices follow the same visible corridor as BFS. A different map can produce a different expansion order.

A* expansion	Removed node (g,h,f)	Priority entries inserted (g,h,f)
1	(1,1) (0,8,8)	(1,2) (1,7,8); (2,1) (1,7,8)
2	(1,2) (1,7,8)	(1,3) (2,6,8)
3	(2,1) (1,7,8)	(3,1) (2,6,8)
4	(1,3) (2,6,8)	(1,4) (3,5,8); (2,3) (3,5,8)
5	(3,1) (2,6,8)	(4,1) (3,5,8)
6	(1,4) (3,5,8)	(1,5) (4,4,8)

For this question, the BFS trace uses a FIFO queue. Manhattan distance does not change the BFS order; if a priority queue is required, that part should be described as A* instead.

2.5 Python BFS implementation

```
from collections import deque
```

```
N = 5
START, GOAL = (1, 1), (5, 5)
BLOCKED = {(2, 2), (2, 4), (3, 2), (4, 4)}
# Deterministic tie-breaking: Right, Down, Left, Up.
MOVES = [(0, 1), (1, 0), (0, -1), (-1, 0)]
```

```
def bfs(start, goal):
    frontier = deque([start])
    parent = {start: None}
    while frontier:
```

```

current = frontier.popleft()
if current == goal:
    break
for dr, dc in MOVES:
    nxt = (current[0] + dr, current[1] + dc)
    inside = 1 <= nxt[0] <= N and 1 <= nxt[1] <= N
    if inside and nxt not in BLOCKED and nxt not in parent:
        parent[nxt] = current
        frontier.append(nxt)
if goal not in parent:
    return None
path, node = [], goal
while node is not None:
    path.append(node)
    node = parent[node]
path.reverse()
return path

path = bfs(START, GOAL)
print(path)
print("cost:", len(path) - 1) # every step costs one

```

```

QUESTION 2 - BREADTH-FIRST SEARCH
Start heuristic h(S): 8
Path: (1, 1) -> (1, 2) -> (1, 3) -> (1, 4) -> (1, 5) -> (2, 5) -> (3, 5) -> (4, 5) -> (5, 5)
Move cost: 1 per step
Optimal path cost: 8
Nodes expanded: 21
BFS frontier: FIFO queue (Manhattan h does not reorder BFS)
>>>

```

Figure 4. Direct Python IDLE 3.12.3 Shell capture after running q2_bfs.py.

3. Partially observable rescue robot

A rescue robot cannot assume it knows the whole building. It senses only adjacent cells, so its internal map is incomplete and can change as it explores. It must not only find a route: it must reason about movement cost, hazards, safety, and new information.

3.1 Constraint analysis and effect on decisions

Given condition	Effect on the agent's decisions
Four-direction movement	The action set is {Up, Down, Left, Right}; diagonal shortcuts are illegal.
Blocked cells	An obstacle is a hard constraint: the robot must not enter it, even if that would be the shortest route.
Adjacent-cell sensing only	The world is partially observable. The robot maintains a belief/known map and cannot plan with certainty over unseen cells.
Base cost 1 per move	Every move increases accumulated path cost g by 1.
Risk surcharge +2	Entering a risky cell costs $1+2=3$. Search must compare cumulative cost, not merely number of steps.
Dynamic knowledge / online search	After sensing or discovering a blockage, update the map and re-plan; a route that was previously preferred may no longer be feasible.
Safety requirement	When information is insufficient, do not blindly commit to an unobserved or known-dangerous cell. Use a safety rule and a conservative unknown-cell

policy.

The key distinction is between “shortest path” and “least-cost path.” With nonuniform step costs, BFS is not generally cost-optimal. Uniform Cost Search (UCS) expands the currently lowest accumulated-cost state and is therefore the natural offline planner when all step costs are nonnegative.

3.2 Sample map and cost comparison

For a concrete demonstration, use the assumptions listed earlier: blocked cells (2,2), (3,2), (4,4); risk cells (1,2), (1,3); and cost 3 to enter a risk cell versus 1 to enter any other free cell.

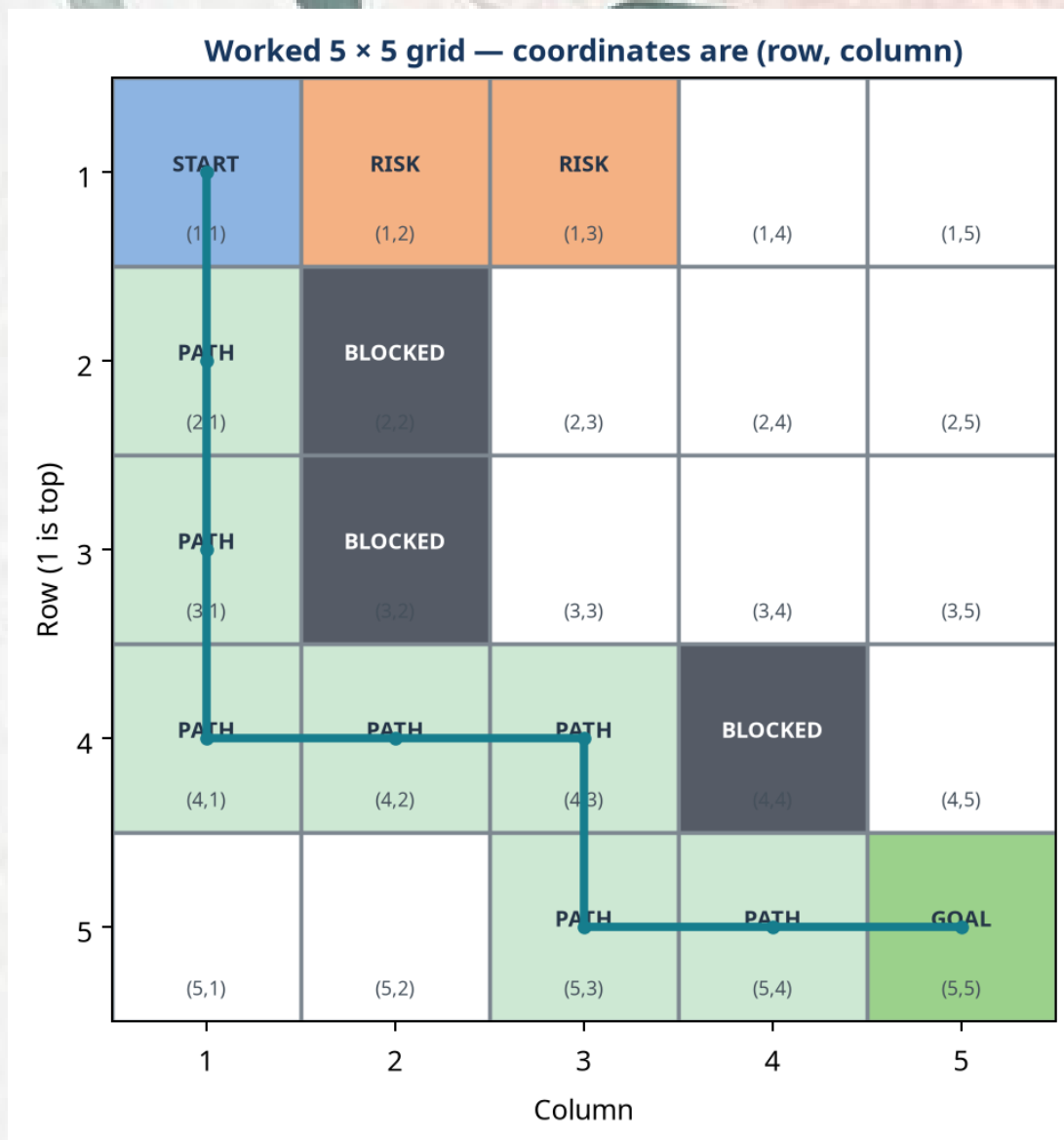


Figure 5. The teal route is the safe minimum-cost route. Orange cells are risky; dark cells are blocked.

Candidate route	Ordinary moves	Risk entries	Total cost

Top corridor: (1,1)→(1,2)→(1,3)→(1,4)→(1,5)→(2,5)→...→(5,5)	8	2	$8 \times 1 + 2 \times 2 = 12$
UCS route: (1,1)→(2,1)→(3,1)→(4,1)→(4,2)→(4,3)→(5,3)→(5,4)→(5,5)	8	0	$8 \times 1 + 0 \times 2 = 8$

The extra-risk charge is added on top of the ordinary move cost. Thus “risk = +2” means a risky entry costs 3 total, not 2. The bottom route has the same number of moves as the illustrated top corridor but avoids the two risk cells, so it is cheaper by 4.

3.3 Uniform Cost Search calculation

Let $g(n)$ be the cheapest known cost from the current planning start to node n . When UCS considers a neighbor v , it computes $g(v) = g(u) + 1 + 2 \cdot I(v \text{ is risky})$, where I is 1 for a risky cell and 0 otherwise. The frontier is a min-priority queue ordered by g ; if a cheaper route to a known cell is discovered, its best cost and parent are updated.

Expansion order	Removed cell and g	New/updated frontier cells (cell: g)
1	11 : 0	12:3, 21:1
2	21 : 1	31:2
3	31 : 2	41:3
4	12 : 3	13:6
5	41 : 3	42:4, 51:4
6	42 : 4	43:5, 52:5
7	51 : 4	—
8	43 : 5	53:6, 33:6
9	52 : 5	—
10	13 : 6	14:7, 23:7
11	33 : 6	34:7
12	53 : 6	54:7
13	14 : 7	15:8, 24:8
14	23 : 7	—
15	34 : 7	35:8
16	54 : 7	55:8
17	15 : 8	25:9
18	24 : 8	—
19	35 : 8	45:9
20	55 : 8	—

The robot stops when G is removed from the min-priority queue—not merely when it is first

generated. With nonnegative costs, that is the point at which its least cost is settled. The selected predecessor chain is $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$.

UCS selects the safe route at cost 8. The alternative across the top also uses 8 moves, but entering its two risky cells adds 2 cost units each, bringing its total to 12.

3.4 Online search: sense, update, re-plan, act

Online rescue-agent loop

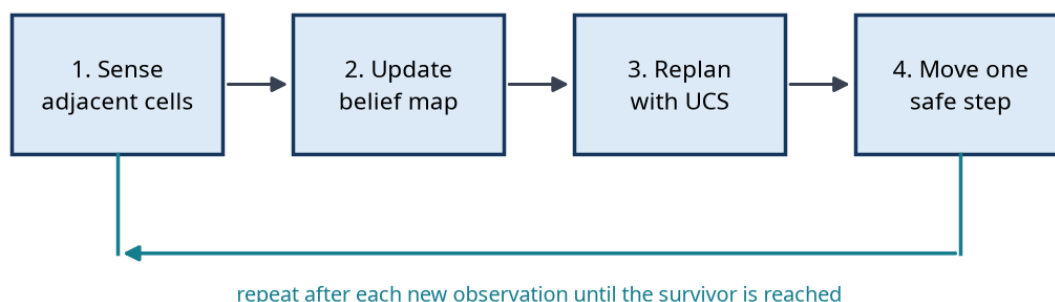


Figure 6. The robot makes one safe move at a time and updates its map after each new observation.

A practical online agent can maintain each cell as “unknown,” “free,” “blocked,” or “risky.” A conservative implementation treats unknown cells as unavailable for the current route, moves to a reachable frontier (a known free cell adjacent to unknown space), senses again, and re-runs UCS. This may require exploration before the goal becomes reachable in the known map. Once enough of the relevant map is known, UCS finds the least-cost route in that known graph.

1. Observe the four adjacent cells and record whether each is free, blocked, or risky.
2. Update the belief map; keep verified information unless a new observation contradicts it.
3. Run UCS through known safe/free cells using move-plus-risk cost. If no route is known, select a reachable frontier.
4. Execute one move only, then sense again; new information can make the old route stale.
5. Stop when the survivor is reached. Otherwise, repeat the cycle.

Treating unknown cells as blocked is cautious, but it can delay the trip while the robot explores. If the robot has a reliable risk model, it could assign unknown cells an expected cost instead; the choice should be made explicit.

3.5 Pseudocode for an online rescue agent

```
known_map = {start: "free"}  
position = start
```

```
while position != goal:
```

```

observations = sense_adjacent(position)
update(known_map, observations)

# Plan only through confirmed free cells; risk changes entry cost.
route = uniform_cost_search(
    start=position,
    goal=goal,
    known_map=known_map,
    step_cost=lambda cell: 1 + (2 if known_map[cell] == "risky" else 0)
)

if route exists:
    next_cell = route[1] # execute one action only
else:
    frontier = reachable_known_frontier(position, known_map)
    if frontier is empty:
        report("No known safe route; request help or new information")
        stop
    next_cell = lowest_cost_frontier_step(position, frontier, known_map)

if known_map[next_cell] == "blocked":
    # Defensive guard: a blocked cell should never be selected.
    continue
move_to(next_cell)
position = next_cell

```

AI concept	Application to this robot
Intelligent agent	The robot perceives adjacent cells, stores an internal state/map, chooses an action, and changes the environment by moving. Its architecture is sensors → belief/map → planner → actuators.
Rationality	A rational action is the best action given what the agent has perceived so far and its stated performance measure. Here the measure is reaching G while avoiding blocked cells and minimizing movement-plus-risk cost. Rationality is relative to available information, not perfect hindsight.
Environment type	Partially observable (limited sensing); sequential (a move affects later choices); deterministic in the sample map (move outcomes fixed); static or slowly changing unless disaster effects move hazards; discrete (grid cells/actions); single-agent in the simplified model; known action rules but initially unknown map.
Online search	The robot interleaves planning and acting because its map is revealed locally. New observations can revise the belief map and prompt re-planning.
Performance measure	Reach G safely; avoid blocked cells; minimize total cost; avoid needless backtracking.

```
report("Survivor reached")
```

A real rescue robot might also use a risk limit or ask for help when it cannot find a safe route. In this example, each risky entry adds 2 cost units, and unseen cells are treated as blocked until the robot has explored them.

3.6 AI concepts in the rescue scenario

3.7 Python UCS implementation for a known map

```

import heapq

N = 5
START, GOAL = (1, 1), (5, 5)
BLOCKED = {(2, 2), (3, 2), (4, 4)}

```

```

RISKY = {(1, 2), (1, 3)}
MOVES = [(0, 1), (1, 0), (0, -1), (-1, 0)]

def ucs(start, goal):
    frontier = [(0, start)]    # (total cost so far, cell)
    best = {start: 0}
    parent = {start: None}
    while frontier:
        cost, current = heapq.heappop(frontier)
        if cost != best[current]: # skip an outdated queue entry
            continue
        if current == goal:
            break
        for dr, dc in MOVES:
            nxt = (current[0] + dr, current[1] + dc)
            inside = 1 <= nxt[0] <= N and 1 <= nxt[1] <= N
            if not inside or nxt in BLOCKED:
                continue
            entry_cost = 1 + (2 if nxt in RISKY else 0)
            new_cost = cost + entry_cost
            if new_cost < best.get(nxt, float("inf")):
                best[nxt] = new_cost
                parent[nxt] = current
                heapq.heappush(frontier, (new_cost, nxt))
    if goal not in parent:
        return None, float("inf")
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    return path, best[goal]

path, cost = ucs(START, GOAL)
print(path)
print("least total cost:", cost)

```

```

>>>
QUESTION 3 - UNIFORM COST SEARCH
Safe least-cost path: (1, 1) -> (2, 1) -> (3, 1) -> (4, 1) -> (4, 2) -> (4, 3) -
> (5, 3) -> (5, 4) -> (5, 5)
Safe route cost: 8
Risky top-route calculation: 8 x 1 + 2 x 2 = 12
Expanded nodes: 20
Obstacle cells are never entered; UCS orders by accumulated cost g.

```

Figure 7. Direct Python IDLE 3.12.3 Shell capture after running q3_ucs.py.